

CM-4 Feature Engineering and Dimension Reduction

3TLDE.A13 - Législation Numérique et Enjeux de l'Analyse des Données

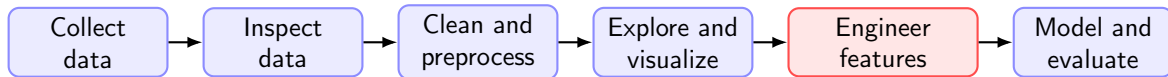


May 28, 2026

Table of contents

- 1 Feature engineering
- 2 Feature creation
- 3 Feature transformation
- 4 Feature selection
- 5 The curse of dimensionality
- 6 Principal Component Analysis
- 7 Summary

Where are we in the data analysis workflow?



In CM-3, we learned how to explore distributions, relationships, and patterns in a dataset.

Now we move one step closer to modeling: we transform raw variables into useful representations.

Today

Feature engineering and dimension reduction help us decide **what information should be given to a model** and **how this information should be represented**.

Learning objectives of CM-4

At the end of this lecture, you should be able to:

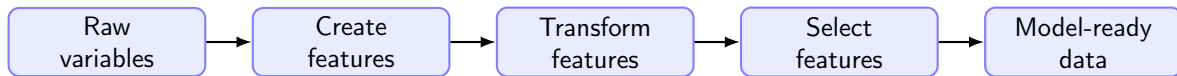
- explain why feature engineering matters before modeling;
- create meaningful numerical, categorical, and date/time features;
- identify irrelevant, redundant, or leaking features;
- distinguish feature engineering, feature selection, and dimension reduction;
- explain the curse of dimensionality intuitively;
- apply and interpret PCA using explained variance;
- understand the main limitations of PCA and other reduction methods.

Table of contents

- 1 Feature engineering
- 2 Feature creation
- 3 Feature transformation
- 4 Feature selection
- 5 The curse of dimensionality
- 6 Principal Component Analysis
- 7 Summary

What is feature engineering?

Feature engineering is the process of creating, transforming, combining, or selecting variables so that they better represent the problem.



Important

Feature engineering is not a purely technical task. It requires data understanding, domain knowledge, and clear assumptions about the analysis goal.

Feature engineering, selection, and reduction

These three terms are related, but they do not mean the same thing.

Concept	Main question	Example
Feature creation	Can we create new useful variables?	<code>total_guests</code>
Feature transformation	Can we change the representation of variables?	<code>log_adr</code>
Feature selection	Which variables should we keep?	remove useless variables
Dimension reduction	Can we use fewer dimensions?	PCA components

Practical message

Feature engineering changes the representation. Feature selection keeps a subset of variables. Dimension reduction creates a compressed representation.

Table of contents

- 1 Feature engineering
- 2 Feature creation**
- 3 Feature transformation
- 4 Feature selection
- 5 The curse of dimensionality
- 6 Principal Component Analysis
- 7 Summary

From raw data to useful features

Raw variables are not always directly useful for analysis or modeling.

Raw variables

- adults
- children
- babies
- stays_in_weekend_nights
- stays_in_week_nights

Engineered features

- total_guests
- has_children
- is_family
- total_nights
- is_long_stay

Core idea

A useful feature should make an important aspect of the problem easier to see, summarize, or model.

A running example: hotel booking data

We continue with a hotel booking dataset.

Each row represents one booking.

Column	Meaning	Type
is_canceled	whether the booking was canceled	target
lead_time	days between booking and arrival	numerical
adr	average daily rate	numerical
adults, children, babies	number of guests	numerical
market_segment	booking channel segment	categorical
deposit_type	deposit policy	categorical

Main modeling question

Can we use booking information to understand or predict cancellation behavior?

Example 1: total number of guests

Some useful features are simple combinations of existing variables.

Raw information

Booking	Adults	Children	Babies
A	2	0	0
B	2	2	0
C	1	0	1

Engineered feature

Booking	Formula	Total guests
A	$2 + 0 + 0$	2
B	$2 + 2 + 0$	4
C	$1 + 0 + 1$	2

Interpretation

`total_guests` directly describes the size of the booking. It is often easier to interpret than three separate variables.

Example 2: total nights and long stays

Some raw variables describe parts of the same concept.

Booking	Weekend nights	Week nights	Total nights	Long stay?	Interpretation
A	0	2	2	No	short stay
B	2	5	7	Yes	one week stay
C	1	0	1	No	one night

Example rule

A booking can be considered a long stay if `total_nights` ≥ 7 .

The threshold is a modeling choice. It should be justified by domain knowledge or exploratory analysis.

Example 3: price per person

A ratio feature can make a numerical variable more comparable across observations.

Booking	ADR	Total guests	Price per person
A	120	2	$120/2 = 60$
B	180	4	$180/4 = 45$
C	90	1	$90/1 = 90$

Interpretation

The raw price adr does not have the same meaning for one guest and for four guests. price_per_person is often more interpretable.

Table of contents

- 1 Feature engineering
- 2 Feature creation
- 3 Feature transformation**
- 4 Feature selection
- 5 The curse of dimensionality
- 6 Principal Component Analysis
- 7 Summary

Why transform numerical variables?

Numerical variables may be difficult to use directly.

Common problems

- highly skewed distribution;
- extreme values;
- variables measured on different scales;
- nonlinear relationship with the target.

Possible transformations

- logarithm;
- standardization;
- normalization;
- binning;
- interaction features.

Key idea

A transformation should simplify the structure of the data, not hide it.

Log transformation: a concrete example

A log transformation is often useful for positive variables with a long right tail.

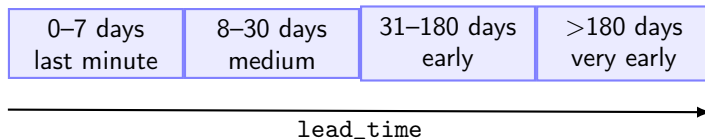
ADR	$\log(ADR)$	$\log(1 + ADR)$
0	undefined	0.00
50	3.91	3.93
100	4.61	4.62
200	5.30	5.30
500	6.21	6.22

Interpretation

The gap between 100 and 500 is much smaller after the log transformation. This reduces the influence of very large values.

Binning: turning numbers into groups

Binning transforms a numerical variable into categories.



Advantages

- easy to interpret;
- captures nonlinear patterns;
- useful for reporting.

Limitations

- loses information;
- thresholds can be arbitrary;
- may create small groups.

Creating bins in pandas

```
1 # Discretize lead_time into categories
2
3 bins = [0, 7, 30, 180, df["lead_time"].max()]
4 labels = [
5     "last_minute",
6     "medium",
7     "early",
8     "very_early"
9 ]
10
11 df["lead_time_group"] = pd.cut(
12     df["lead_time"],
13     bins=bins,
14     labels=labels,
15     include_lowest=True
16 )
```

After binning

Check the number of observations in each bin.

```
1 df["lead_time_group"].value_counts()
```

Categorical feature engineering

Categorical variables often need special treatment before modeling.

Situation	Method	Example
No natural order	one-hot encoding	market_segment
Natural order	ordinal encoding	low, medium, high
Many rare categories	group rare levels	replace by Other
High cardinality	frequency or target encoding	postal code, product id

Important

Do not use ordinal encoding for a variable without order, unless the model and interpretation justify it.

One-hot encoding: example

For a categorical variable without order, one-hot encoding creates binary indicators.

deposit_type	No Deposit	Non Refund	Refundable
No Deposit	1	0	0
Non Refund	0	1	0
Refundable	0	0	1

Consequence

One categorical variable can become many binary variables. This increases the dimensionality of the data.

One-hot encoding in pandas

```
1 cat_cols = [  
2     "market_segment",  
3     "deposit_type",  
4     "customer_type"  
5 ]  
6  
7 df_encoded = pd.get_dummies(  
8     df,  
9     columns=cat_cols,  
10    drop_first=False  
11 )  
12  
13 print(df_encoded.shape)
```

Interpretation

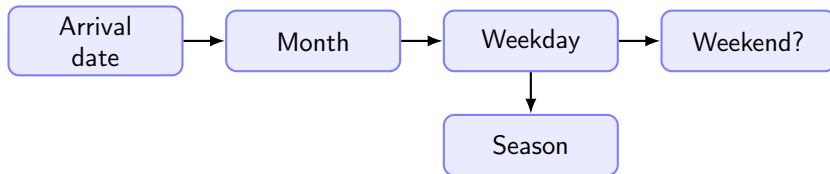
The number of columns increases after one-hot encoding. This is often necessary, but it can also make the dataset high-dimensional.

Connection

This is one reason why feature selection and dimension reduction may become useful.

Date and time feature engineering

Date variables are rarely used directly. We usually extract useful components.



Examples

From an arrival date, we can create `arrival_month`, `arrival_weekday`, `is_weekend_arrival`, and `season`.

Creating date/time features

```
1 # Build an arrival date
2
3 df["arrival_date"] = pd.to_datetime(
4     df["arrival_date_year"].astype(str) + "-" +
5     df["arrival_date_month"].astype(str) + "-" +
6     df["arrival_date_day_of_month"].astype(str)
7 )
8
9 # Extract useful information
10
11 df["arrival_weekday"] = df["arrival_date"].dt.day_name()
12 df["arrival_month_num"] = df["arrival_date"].dt.month
13
14 df["is_weekend_arrival"] = (
15     df["arrival_date"].dt.weekday >= 5
16 ).astype(int)
```

Interpretation

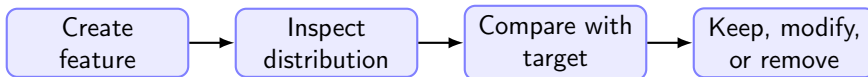
Date features can capture seasonality, weekly patterns, and holiday effects.

Warning

Only use information that would be available at prediction time.

Feature engineering should be checked by EDA

After creating a feature, we should inspect it.



Example

After creating `lead_time_group`, compare the cancellation rate across groups. If the groups behave differently, the feature may be useful.

Table of contents

- 1 Feature engineering
- 2 Feature creation
- 3 Feature transformation
- 4 Feature selection**
- 5 The curse of dimensionality
- 6 Principal Component Analysis
- 7 Summary

Why feature selection?

More features are not always better.

Too many features can cause

- noise;
- redundancy;
- slower computation;
- lower interpretability;
- higher risk of overfitting.

Feature selection helps

- simplify the dataset;
- focus on useful variables;
- remove irrelevant variables;
- improve communication;
- prepare modeling.

Key message

A simpler feature set is often easier to understand and more robust.

Basic feature selection methods

For this course, we focus on simple and interpretable methods.

Method	What it removes	Example
Constant feature check	no information	same value for all rows
Missing value check	unreliable variables	95% missing values
Correlation check	redundancy	two nearly identical variables
Target relationship	weak relevance	no visible relation to target
Leakage check	invalid features	future information

Important

Feature selection should not be only automatic. It should also use domain knowledge and the prediction scenario.

Removing constant or near-constant features

A feature with almost no variation cannot explain much.

Feature	Row 1	Row 2	Row 3	Row 4	Row 5
country	PRT	PRT	PRT	PRT	PRT
hotel	City	Resort	City	City	Resort

Interpretation

If all rows come from the same country, country cannot help distinguish different bookings in this dataset.

Checking low-variance features

```
1 # Number of unique values per column
2
3 n_unique = df.nunique().sort_values()
4 print(n_unique.head(10))
5
6 # Percentage of the most frequent value
7
8 dominance = df.apply(
9     lambda col: col.value_counts(normalize=True).iloc[0]
10 )
11
12 print(dominance.sort_values(ascending=False).head(10))
```

Rule of thumb

If one value represents more than 95% or 99% of the rows, the feature may be weakly informative.

But

A rare value can still be important if it corresponds to a meaningful event.

Redundant features: correlation example

Two numerical variables may contain almost the same information.

Booking	Weekend nights	Week nights	Total nights
A	0	2	2
B	2	5	7
C	1	4	5
D	0	1	1

Interpretation

`total_nights` is computed from the two original variables. Keeping all three may be useful, but it also creates redundancy.

Correlation check

```
1 num_cols = [  
2     "lead_time",  
3     "adr",  
4     "total_guests",  
5     "total_nights",  
6     "price_per_person"  
7 ]  
8  
9 corr = df[num_cols].corr()  
10 print(corr)
```

Interpretation

A strong correlation may indicate redundancy, but it does not automatically mean that one feature must be removed.

A practical feature selection checklist

Before modeling, ask the following questions.

- ❶ Is the feature available at prediction time?
- ❷ Does the feature contain many missing values?
- ❸ Is the feature constant or almost constant?
- ❹ Is the feature redundant with another feature?
- ❺ Is the feature interpretable or useful for the objective?
- ❻ Does EDA suggest a relationship with the target?

Key message

Feature selection should protect the validity of the analysis, not only reduce the number of columns.

Table of contents

- 1 Feature engineering
- 2 Feature creation
- 3 Feature transformation
- 4 Feature selection
- 5 The curse of dimensionality**
- 6 Principal Component Analysis
- 7 Summary

Why high dimensionality is a problem

High-dimensional data appears in many situations:

- one-hot encoding creates many binary variables;
- text data can have thousands of word features;
- images contain many pixels;
- repeated feature engineering creates many combinations.

Curse of dimensionality

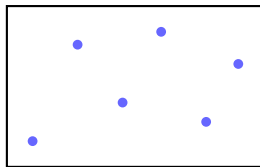
As the number of features increases, data points become sparse, distances become less meaningful, and models often need much more data to generalize well.

A simple intuition: covering a space

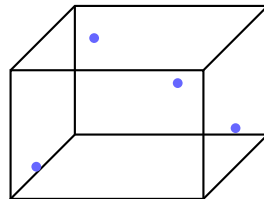
Imagine that we want enough data to cover the feature space.



1D: points can cover a line



2D: same number looks sparse



3D+: even sparser

Interpretation

With more dimensions, the same number of observations covers a much larger space. This makes learning patterns harder.

Concrete calculation: grid cells

Suppose each feature is divided into 10 intervals.

Number of features	Cells per feature	Total cells
1	10	10
2	10	100
3	10	10^3
10	10	10^{10}
50	10	10^{50}

Interpretation

The number of possible regions grows exponentially with the number of features. This is why high-dimensional data can require much more data.

Feature selection versus dimension reduction

There are two broad strategies for reducing complexity.

Feature selection

Keep a subset of the original variables.

- Easy to interpret.
- Keeps original meaning.
- Example: keep `lead_time`, remove leakage.

Dimension reduction

Create new lower-dimensional variables.

- Can compress information.
- New variables may be hard to interpret.
- Example: PCA components.

Practical choice

If interpretability is central, start with feature selection. If compression or visualization is central, dimension reduction may be useful.

Table of contents

- 1 Feature engineering
- 2 Feature creation
- 3 Feature transformation
- 4 Feature selection
- 5 The curse of dimensionality
- 6 Principal Component Analysis**
- 7 Summary

PCA: what problem are we solving?

PCA is not just a plotting tool. It answers a precise question.

Main question

Can we replace p original numerical variables by $k < p$ new variables while **keeping as much information as possible**?

Original representation

Each observation is described by

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip}).$$

The data matrix is

$$X \in \mathbb{R}^{n \times p}.$$

PCA representation

PCA creates new variables

$$\mathbf{z}_i = (z_{i1}, z_{i2}, \dots, z_{ik}),$$

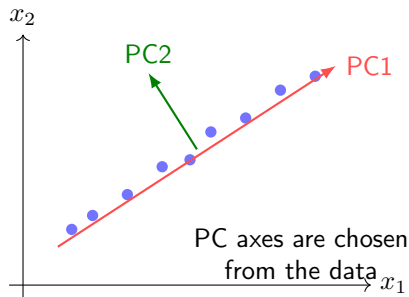
called **principal component scores**. They are linear combinations of the original variables.

Key idea

PCA chooses the new axes so that the projected data has maximum variance.

PCA is a change of coordinate system

PCA does not select original features. It creates a **new coordinate system**.



Important distinction

A principal component is a **direction**.

A PCA score is the **coordinate of one observation** on that direction.

Linear combinations: components and scores

Let $\mathbf{w}_1^\top = (w_{11}, w_{12}, \dots, w_{1p})$ be the first PCA direction.

First component direction

$$\mathbf{w}_1 = \begin{bmatrix} w_{11} \\ w_{12} \\ \vdots \\ w_{1p} \end{bmatrix}, \quad \|\mathbf{w}_1\| = 1.$$

The constraint $\|\mathbf{w}_1\| = 1$ avoids arbitrarily increasing the variance by multiplying the vector.

Score of observation i

$$z_{i1} = \mathbf{x}_i^\top \mathbf{w}_1$$

$$z_{i1} = w_{11}x_{i1} + \dots + w_{1p}x_{ip}.$$

Vocabulary

The coefficients w_{1j} are often called **loadings**. The values z_{i1} are the **scores**.

Mean-centering and standardization before PCA

Before PCA, we usually center each numerical variable and put them on comparable scales.

Mean-centering

$$x_{ij}^c = x_{ij} - \bar{x}_j$$

- Almost always needed.
- The centered data matrix is denoted by X_c .

Standardization

$$x_{ij}^s = \frac{x_{ij} - \bar{x}_j}{s_j}$$

- Usually recommended when units differ.
- Equivalent to PCA on the correlation matrix.

Practical rule

For this course, standardize numerical features before PCA unless all variables have the same unit and comparable variance. In `sklearn`, PCA centers the data but does not scale it.

Why scaling matters: a concrete example

Suppose we use two variables:

$$V_1 \in [0, 500], \quad V_2 \in [1, 20].$$

Without scaling

The covariance of `adr` can dominate because it has larger numerical values.

$$\text{Var}(V_1) \gg \text{Var}(V_2)$$

PC1 may mostly become a V_1 axis.

After standardization

Both variables have mean 0 and variance 1.

$$\text{Var}(V_1^s) = \text{Var}(V_2^s) = 1$$

PCA focuses on their correlation structure.

Message

PCA is sensitive to scale because it is based on variance.

The PCA objective for the first component

After centering or standardizing the data, PCA looks for a unit vector $\mathbf{w}_1$.

Projection of the data

The projected values are

$$\mathbf{z}_1 = X_c \mathbf{w}_1.$$

Their sample variance is

$$\text{Var}(\mathbf{z}_1) = \frac{1}{n-1} \mathbf{z}_1^\top \mathbf{z}_1.$$

Since $\mathbf{z}_1 = X_c \mathbf{w}_1$:

$$\text{Var}(\mathbf{z}_1) = \frac{1}{n-1} \mathbf{w}_1^\top X_c^\top X_c \mathbf{w}_1 = \mathbf{w}_1^\top S \mathbf{w}_1,$$

where

$$S = \frac{1}{n-1} X_c^\top X_c$$

is the covariance matrix.

Optimization problem behind PC1

The first principal component solves:

$$\max_{\mathbf{w}_1} \mathbf{w}_1^\top S \mathbf{w}_1$$

subject to

$$\mathbf{w}_1^\top \mathbf{w}_1 = 1.$$

Meaning

Find the direction where the projected data has the largest variance.

Constraint

The vector must have norm 1, otherwise the variance can be increased just by multiplying $\mathbf{w}_1$.

Core idea

PCA does not choose arbitrary linear combinations. It chooses the linear combinations that solve this variance-maximization problem.

Why eigenvectors appear

To solve

$$\max_{\mathbf{w}_1} \quad \mathbf{w}_1^\top S \mathbf{w}_1 \quad \text{subject to} \quad \mathbf{w}_1^\top \mathbf{w}_1 = 1,$$

we use a Lagrange multiplier:

$$\mathcal{L}(\mathbf{w}_1, \lambda) = \mathbf{w}_1^\top S \mathbf{w}_1 - \lambda(\mathbf{w}_1^\top \mathbf{w}_1 - 1).$$

Taking the derivative with respect to $\mathbf{w}_1$ gives:

$$2S\mathbf{w}_1 - 2\lambda\mathbf{w}_1 = 0.$$

Therefore:

$$S\mathbf{w}_1^* = \lambda_1\mathbf{w}_1^*.$$

Conclusion

The first PCA direction is the eigenvector of the covariance matrix S associated with its largest eigenvalue λ_1 .

Second principal component

PC1 captures the largest possible variance.

PC2 captures the largest remaining variance, subject to being orthogonal to PC1.

Optimization problem

$$\max_{\mathbf{w}_2} \quad \mathbf{w}_2^\top S \mathbf{w}_2$$

subject to

$$\mathbf{w}_2^\top \mathbf{w}_2 = 1, \quad \mathbf{w}_2^\top \mathbf{w}_1 = 0.$$

Therefore,

$$S \mathbf{w}_2^* = \lambda_2 \mathbf{w}_2^*,$$

where $\mathbf{w}_2^*$ is the eigenvector corresponding to the second largest eigenvalue λ_2 of S .

General rule for PCA components

Let the eigenvalues of S be ordered as

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p.$$

The j -th principal component direction is

$$S\mathbf{w}_j = \lambda_j\mathbf{w}_j.$$

General rule

PC j is the direction with the largest possible remaining variance, while being orthogonal to all previous components:

$$\mathbf{w}_j^\top \mathbf{w}_k = 0, \quad k < j.$$

Eigenvalues and explained variance

For each principal component:

$$S\mathbf{w}_j = \lambda_j \mathbf{w}_j, \quad j = 1, \dots, p.$$

Eigenvector $\mathbf{w}_j$

- Gives the direction of component j .
- Defines the linear combination of original variables.
- Components are orthogonal to each other.

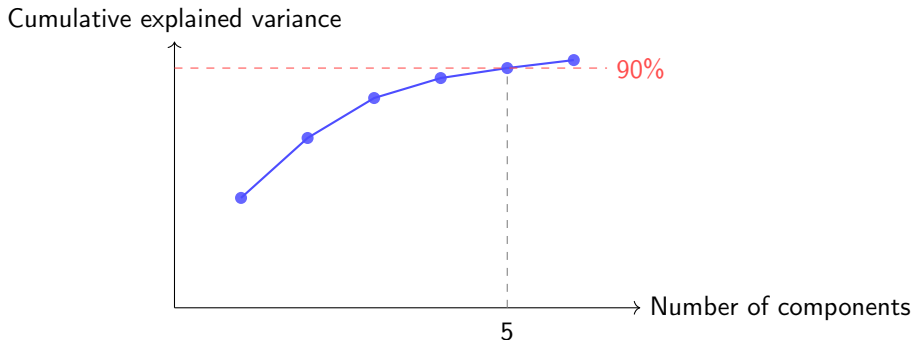
Eigenvalue λ_j

- Gives the variance captured by component j .
- Larger eigenvalue means more information kept.
- Components are ordered by decreasing eigenvalue.

$$\text{Explained variance ratio of PC } j = \frac{\lambda_j}{\lambda_1 + \lambda_2 + \dots + \lambda_p}.$$

Choosing k : cumulative explained variance

A common strategy is to choose the smallest number of components that keeps enough variance.



Rule of thumb

The best value of k depends on the goal: visualization, compression, noise reduction, or modeling.

Matrix formulation

Let W_k contain the first k eigenvectors:

$$W_k = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_k] \in \mathbb{R}^{p \times k}.$$

Projection to PCA space

$$Z_k = X_c W_k.$$

Here $Z_k \in \mathbb{R}^{n \times k}$ contains the PCA scores.

Approximate reconstruction

$$\hat{X}_c = Z_k W_k^\top.$$

Adding the mean back gives

$$\hat{X} = \hat{X}_c + \boldsymbol{\mu}.$$

Another view: PCA minimizes reconstruction error

PCA can be explained in two equivalent ways.

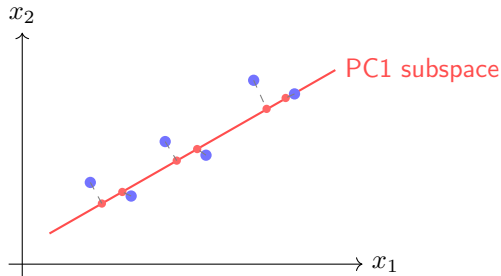
Variance view

Choose the k -dimensional subspace where the projected data has maximum variance.

Reconstruction view

Choose the k -dimensional subspace that minimizes the squared reconstruction error:

$$\|X_c - \hat{X}_c\|_F^2.$$



Example: two standardized variables

Suppose we have two standardized variables:

- V_1 : standardized lead_time;
- V_2 : standardized total_nights.

Assume their covariance matrix is:

$$S = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$

Variance and correlation

Both variables have a variance 1 because they are standardized.

The covariance 0.8 means the two variables are strongly positively correlated.

Computing eigenvectors in the 2D example

For

$$S = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix},$$

the eigenvectors are:

$$\mathbf{w}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.71 \\ 0.71 \end{bmatrix}, \quad \lambda_1 = 1.8.$$

$$\mathbf{w}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \approx \begin{bmatrix} -0.71 \\ 0.71 \end{bmatrix}, \quad \lambda_2 = 0.2.$$

Interpretation

PC1 follows the common direction where both variables increase together. PC2 captures the contrast between them.

The resulting principal components

The PCA directions give the formulas for the new variables.

First component

$$PC1 = 0.71x_1 + 0.71x_2$$

This component is high when both variables are high.

Second component

$$PC2 = -0.71x_1 + 0.71x_2$$

This component is high when x_2 is high compared with x_1 .

Concrete calculation for one observation

Consider one standardized observation:

$$x_1 = 1.2, \quad x_2 = 0.4.$$

Using

$$PC1 = 0.71x_1 + 0.71x_2, \quad PC2 = -0.71x_1 + 0.71x_2,$$

we obtain:

$$PC1 = 0.71(1.2) + 0.71(0.4) = 1.136,$$

$$PC2 = -0.71(1.2) + 0.71(0.4) = -0.568.$$

Meaning

The original point $(1.2, 0.4)$ is represented by the new coordinates $(1.136, -0.568)$ in the PCA coordinate system.

Explained variance in the example

The total variance is the sum of the eigenvalues:

$$\lambda_1 + \lambda_2 = 1.8 + 0.2 = 2.$$

PC1

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{1.8}{2} = 0.90.$$

PC1 keeps 90% of the variance.

PC2

$$\frac{\lambda_2}{\lambda_1 + \lambda_2} = \frac{0.2}{2} = 0.10.$$

PC2 keeps 10% of the variance.

Dimension reduction decision

If we keep only PC1, we reduce the data from 2 dimensions to 1 while keeping 90% of the variance.

Example of explained variance table

Each principal component explains a part of the total variance.

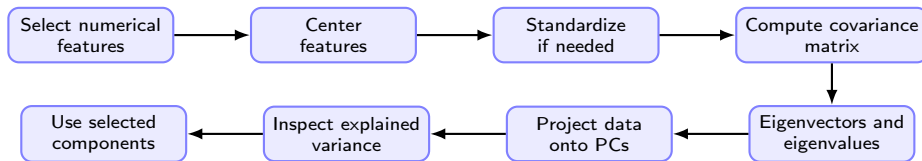
Component	Eigenvalue	Explained variance ratio	Cumulative ratio
PC1	2.10	0.42	0.42
PC2	1.25	0.25	0.67
PC3	0.75	0.15	0.82
PC4	0.40	0.08	0.90
PC5	0.20	0.04	0.94

Interpretation

The first 4 components keep about 90% of the variance. We may reduce the dimension from 5 features to 4 components.

PCA workflow

A typical PCA workflow has several steps.



Important

In practice, libraries compute PCA efficiently using numerical linear algebra, often through SVD rather than explicitly computing all eigenvectors.

Applying PCA in Python

```
1 from sklearn.preprocessing import StandardScaler
2 from sklearn.decomposition import PCA
3
4 num_cols = [
5     "lead_time",
6     "adr",
7     "total_guests",
8     "total_nights",
9     "price_per_person"
10 ]
11
12 X_num = df[num_cols].dropna()
13
14 # sklearn PCA centers data, but does not scale it.
15 scaler = StandardScaler()
16 X_scaled = scaler.fit_transform(X_num)
17
18 pca = PCA(n_components=2)
19 X_pca = pca.fit_transform(X_scaled)
20
21 print(pca.components_)           # loadings
22 print(pca.explained_variance_)   # eigenvalues
23 print(pca.explained_variance_ratio_) # ratios
```

Output

components_ contains the PCA directions.

Scores

X_pca contains the coordinates of observations in the PCA space.

Variance

explained_variance_ corresponds to eigenvalues.

Plotting explained variance

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 pca_full = PCA()
5 X_pca_full = pca_full.fit_transform(X_scaled)
6
7 cum_var = np.cumsum(
8     pca_full.explained_variance_ratio_
9 )
10
11 plt.plot(range(1, len(cum_var) + 1), cum_var, marker="o")
12 plt.axhline(0.90, linestyle="--")
13 plt.xlabel("Number of components")
14 plt.ylabel("Cumulative explained variance")
15 plt.show()
```

Interpretation

The curve helps decide how many components are needed to preserve most of the variance.

Using PCA for visualization

One common use of PCA is to visualize high-dimensional data in 2D.

Typical interpretation questions

- Do canceled and non-canceled bookings form separate groups?
- Are there outliers?
- Are some segments clustered together?
- Is the 2D projection informative enough?

Caution

A PCA plot is a projection. Separation in 2D is useful, but lack of separation does not prove that no pattern exists in higher dimensions.

Limitations of PCA

PCA is useful, but it is not a universal solution.

- PCA is linear: it captures linear directions of variance.
- PCA is unsupervised: it does not use the target variable.
- Maximum variance does not always mean maximum predictive power.
- Components are combinations of variables and may be hard to interpret.
- PCA can be sensitive to outliers.
- PCA usually requires numerical, centered, and often standardized features.

Practical message

Use PCA when compression or visualization is useful, but do not blindly replace interpretable features by principal components.

Beyond PCA

Other dimension reduction methods exist, especially for visualization.

Method	Main use	Caution
t-SNE	nonlinear visualization	sensitive to parameters
UMAP	nonlinear visualization	distances are not always interpretable
Autoencoder	learned compressed representation	requires neural networks

For this course

PCA is the main method to understand. t-SNE, UMAP, and autoencoders are useful extensions, but they require more advanced modeling choices.

Table of contents

- 1 Feature engineering
- 2 Feature creation
- 3 Feature transformation
- 4 Feature selection
- 5 The curse of dimensionality
- 6 Principal Component Analysis
- 7 Summary**

Key takeaways

- ➊ Feature engineering transforms raw data into more useful representations.
- ➋ Useful features often come from domain knowledge and EDA.
- ➌ Numerical, categorical, and date/time variables require different transformations.
- ➍ More features are not always better: redundancy, noise, and leakage matter.
- ➎ High-dimensional data can become sparse and difficult to model.
- ➏ PCA creates new components that capture variance in fewer dimensions.
- ➐ PCA is useful for visualization and compression, but can reduce interpretability.